

Agentic AI Essentials

 Demo-1

Demo: Creating Python virtual environment on VS Code

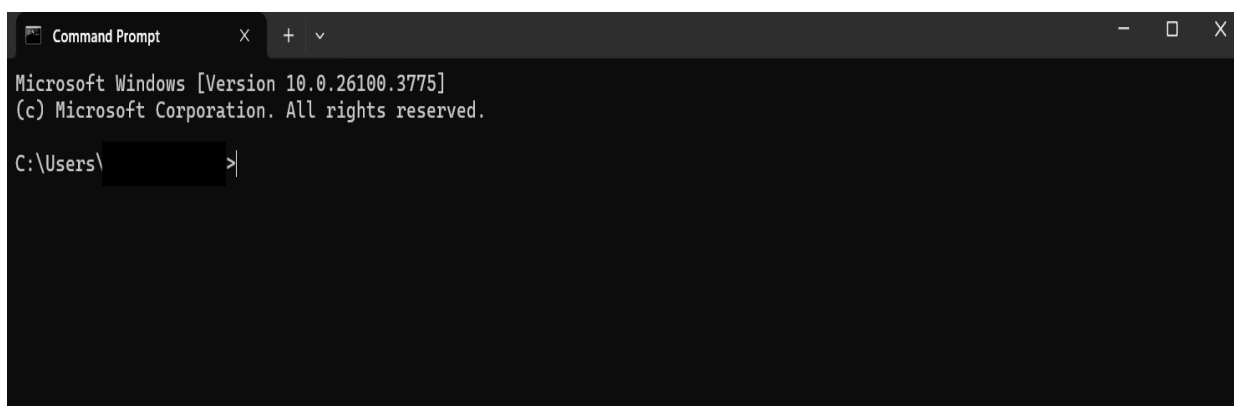
Why Use a Virtual Environment?

- Averts clashes among various project dependencies. Every project may utilize its unique collection of packages without affecting other projects or the global Python installation on the system.
- Make certain that every required package and its respective versions are documented in the environment folder as it makes it easier to share and replicate projects, as all dependencies are clearly defined.

Step-by-step guide for creating and virtual environment on VS Code:

Step 1: Open Command Prompt (CMD)

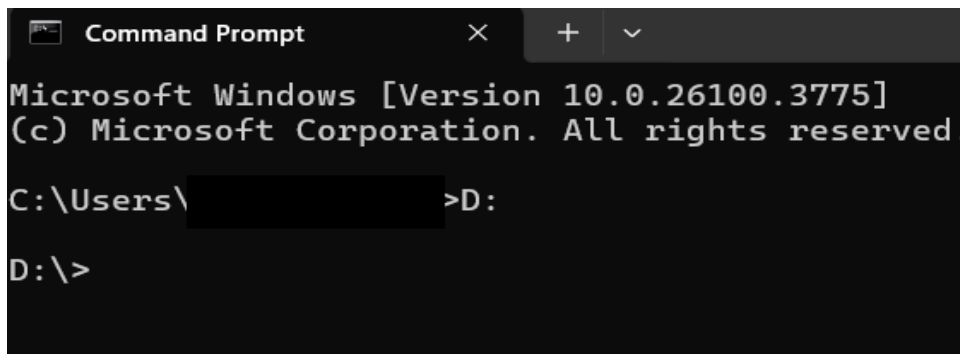
- Start with a clean Windows Command Prompt.
- The CMD is used for setting up project folders and creating the environment.



Step 2: Navigate to the Desired Drive

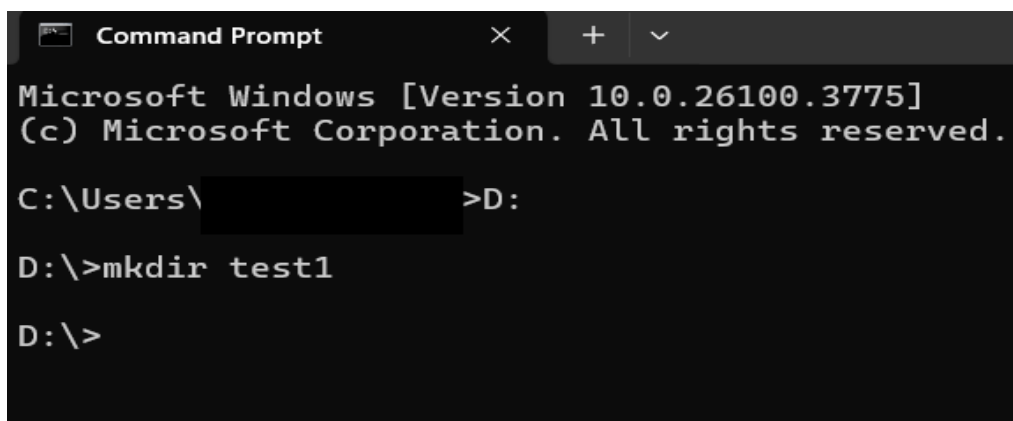
- Change the current drive to 'D:' using the command `D :`

- You can use the drive you want to create your folder or virtual environment in.



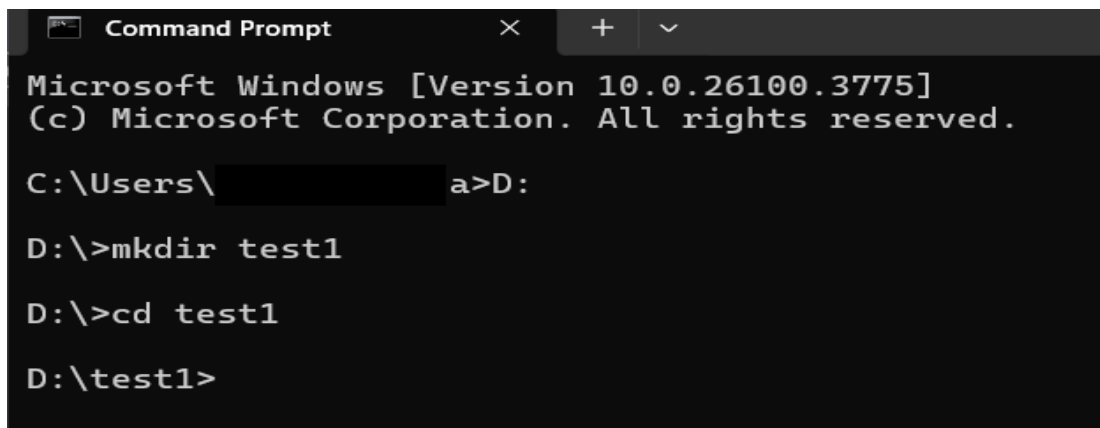
```
Command Prompt
Microsoft Windows [Version 10.0.26100.3775]
(c) Microsoft Corporation. All rights reserved.
C:\Users\[redacted]>D:
D:\>
```

Step 3: Create a project folder named "test1" using the command `mkdir test1`.



```
Command Prompt
Microsoft Windows [Version 10.0.26100.3775]
(c) Microsoft Corporation. All rights reserved.
C:\Users\[redacted]>D:
D:\>mkdir test1
D:\>
```

Step 4: Enter the project folder using the command `cd test1`.



```
Microsoft Windows [Version 10.0.26100.3775]
(c) Microsoft Corporation. All rights reserved.

C:\Users\>D:

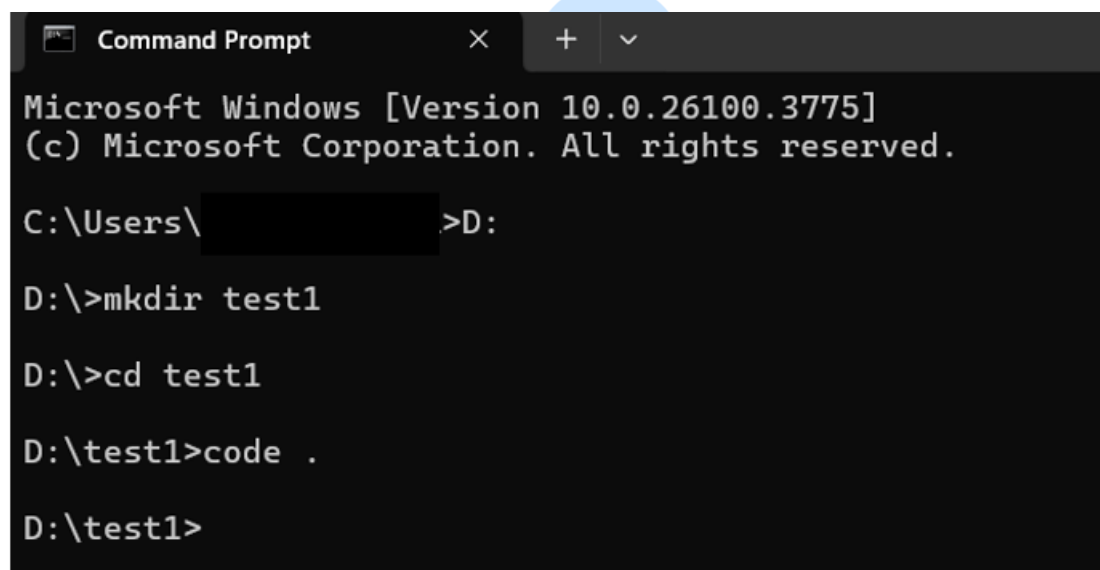
D:\>mkdir test1

D:\>cd test1

D:\test1>
```

Step 5:

- Launch VS Code from the "test1" directory using the command '`code .`'
- This sets "test1" as the root directory and all future files are stored.
- You can do the same by directly creating folder from VS Code interface.



```
Microsoft Windows [Version 10.0.26100.3775]
(c) Microsoft Corporation. All rights reserved.

C:\Users\>D:

D:\>mkdir test1

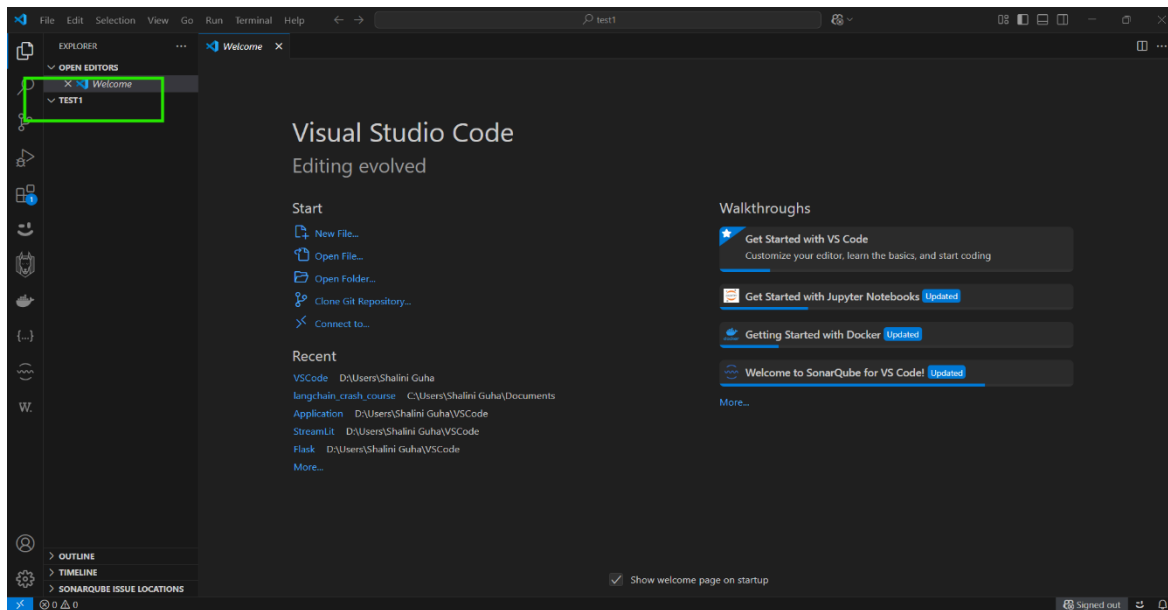
D:\>cd test1

D:\test1>code .

D:\test1>
```

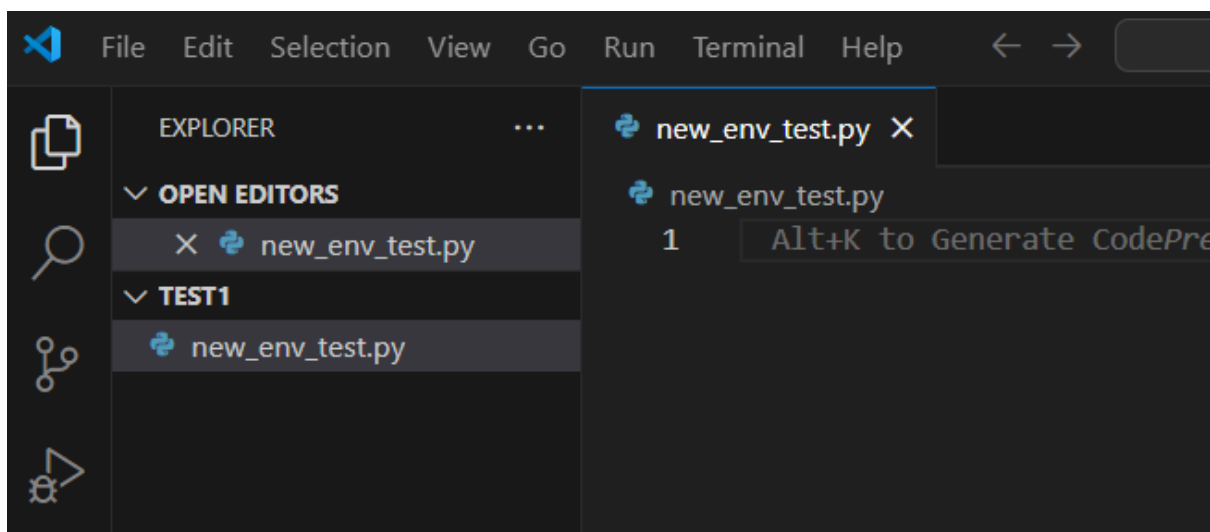
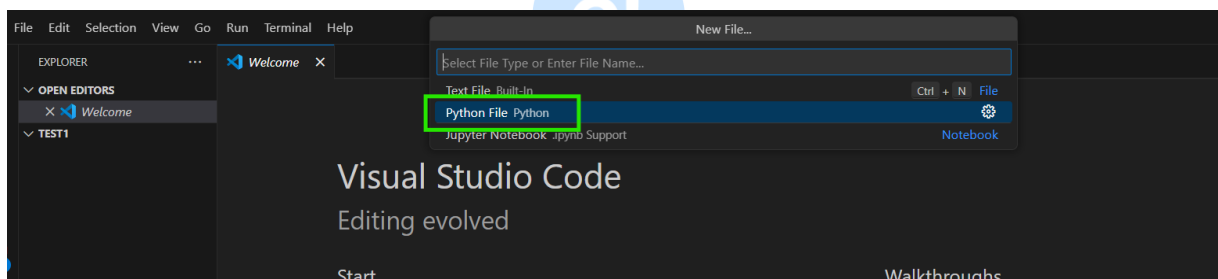
Step 6: VS Code Interface

Check the Explorer in VS Code has the "test1" as the folder name to verify that VS Code recognizes "test1" as the project and working directory.



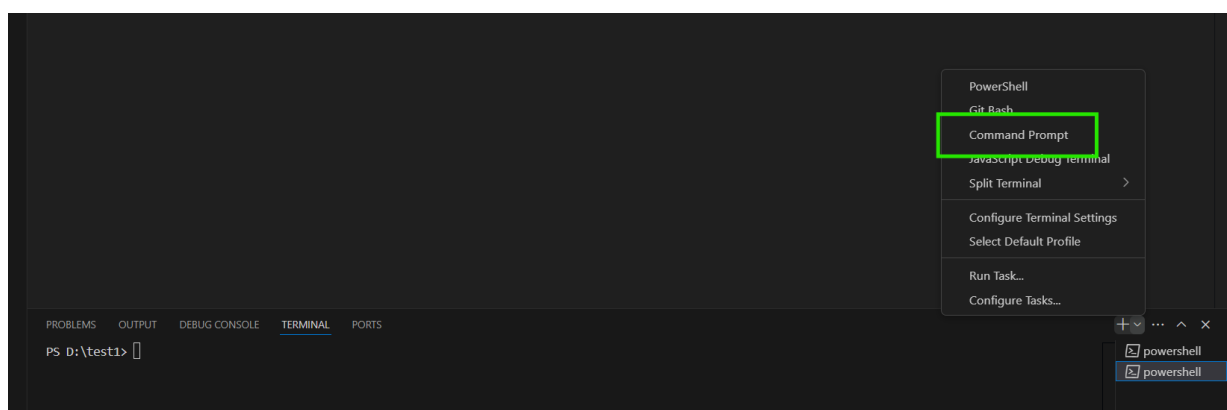
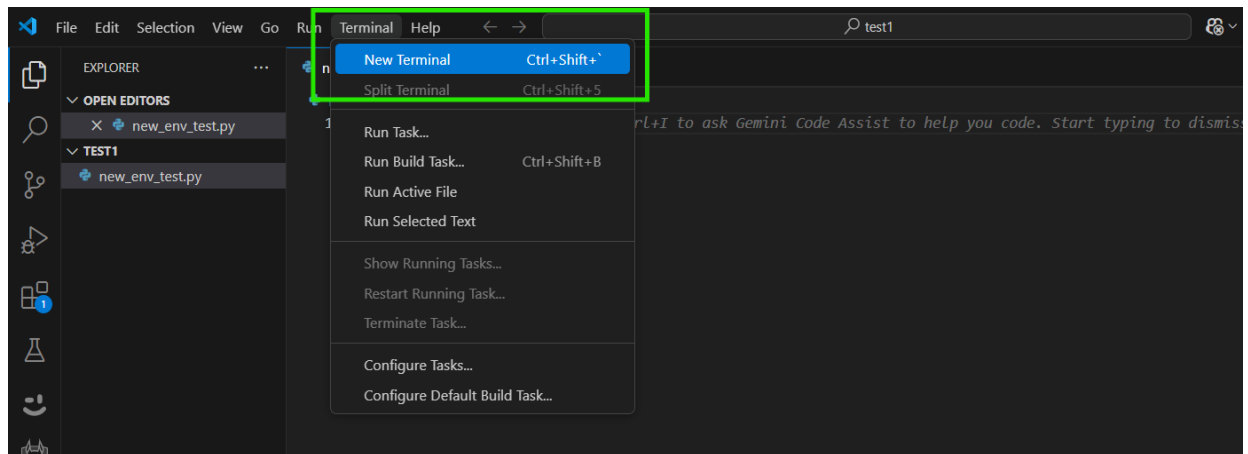
Step 7: Create Python file

In VS code create a new Python file (e.g. `new_env_test.py`)



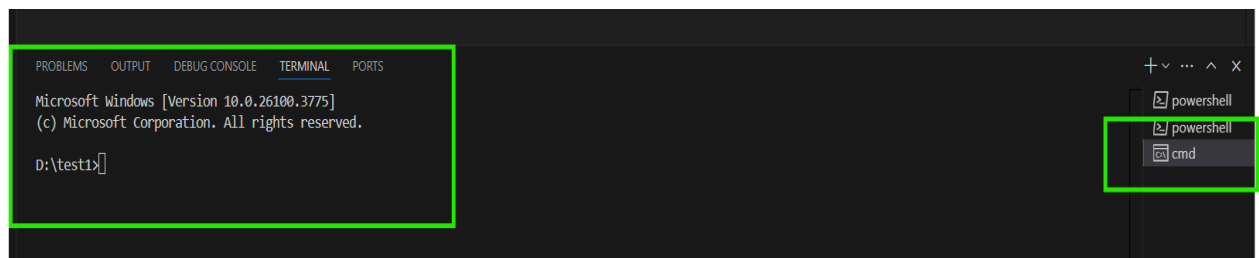
Step 8: Open Command Prompt Terminal

- Open a new command prompt (cmd) terminal in VS Code.
- VS code automatically opens powershell so you have to click on cmd and it is critical to create and activate in CMD.



Command Prompt Terminal Ready:

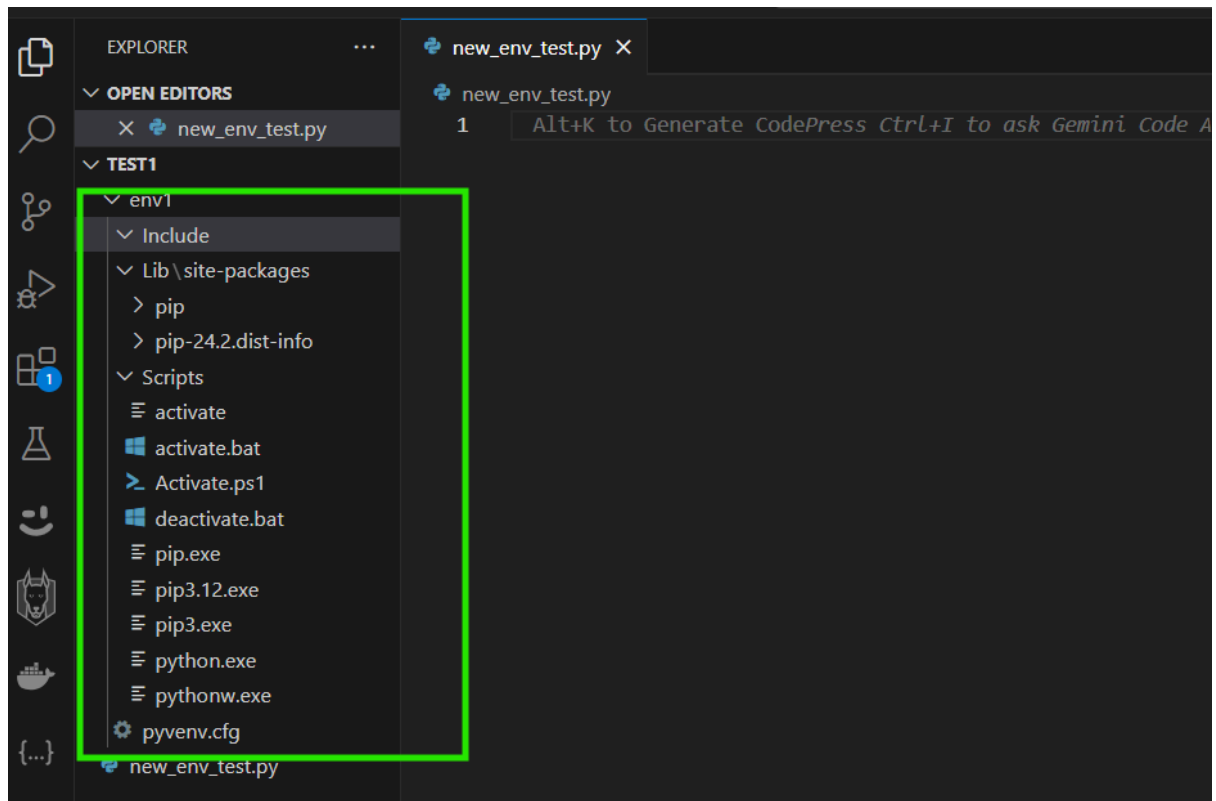
The Windows Command Prompt (cmd) is now open within VS Code's terminal panel, with the current directory set to `D:\test1` which is the selected environment for running the virtual environment commands.



Step 9: Create Virtual Environment

- Execute the command `python -m venv env1` within the VS Code terminal.
- This creates the "env1" directory containing the virtual environment's files.

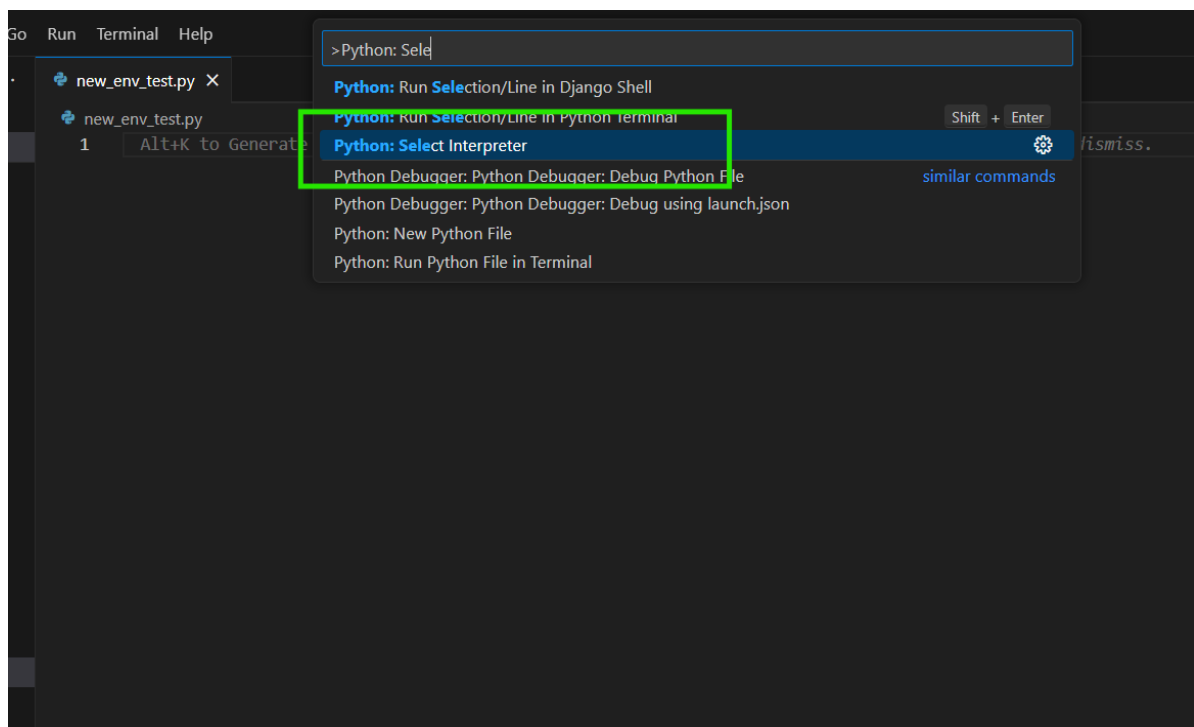
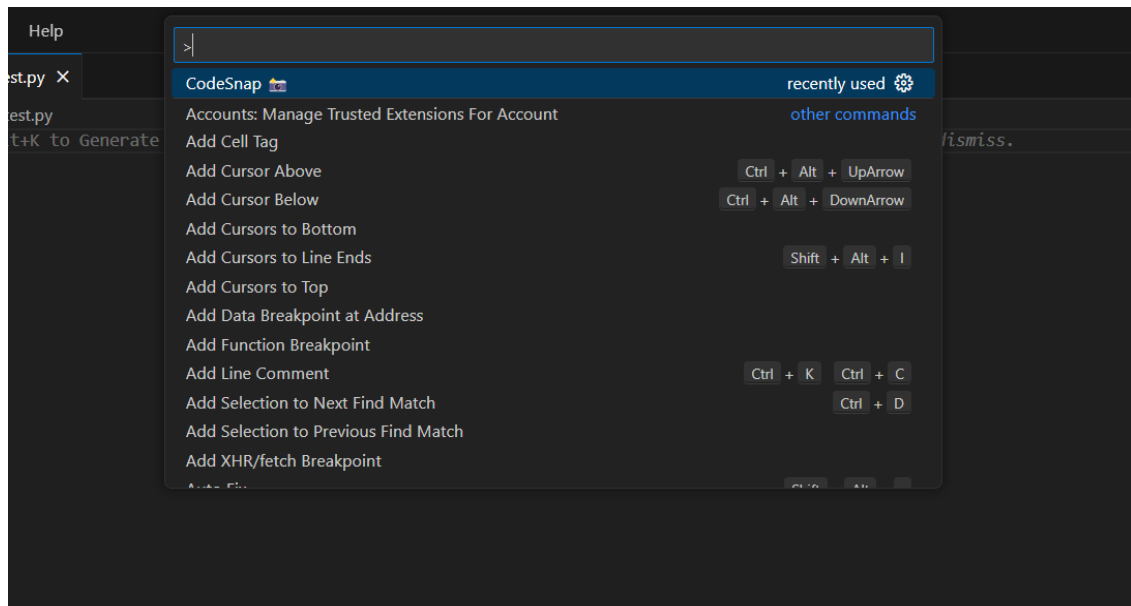


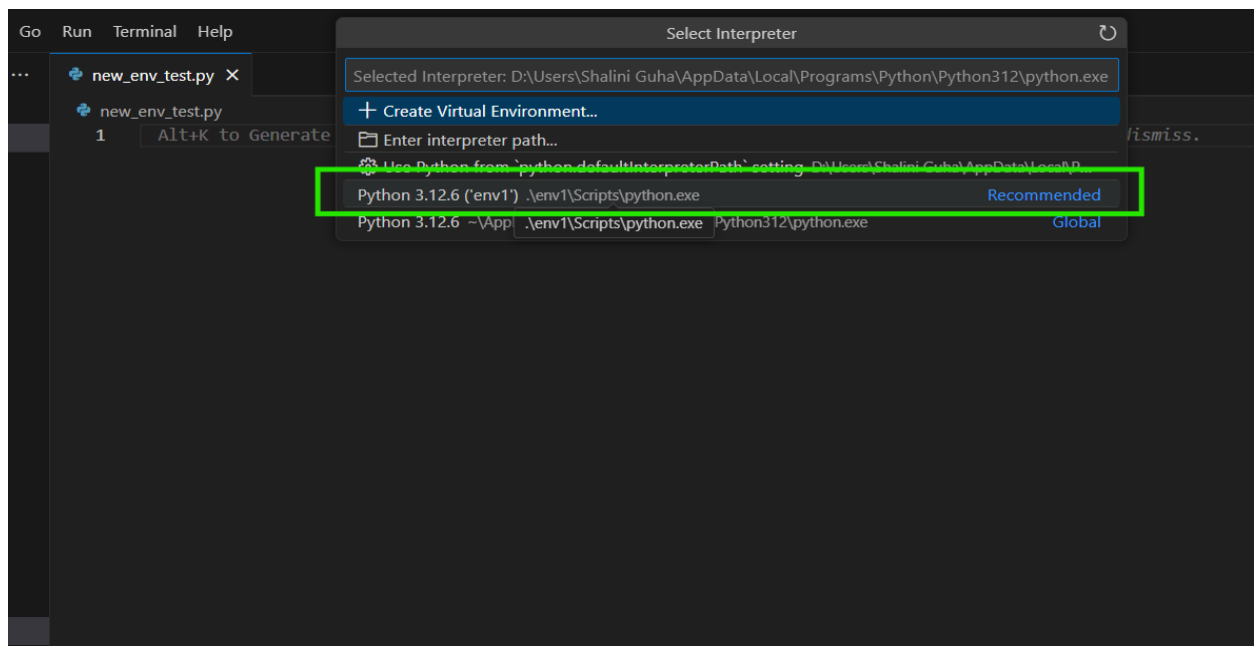


Step 10: Select Python Interpreter



- Open the Command Palette (Ctrl+Shift+P) and search for "Python: Select Interpreter."
- Select the Python interpreter located within your environment folder (e.g., ".../env1/Scripts/python.exe").
- This ensures that VS Code will run your Python code and manage packages within the environment.





Step 11: Running code in Python (Powershell)

```
def add(x, y):  
    return x + y  
  
def subtract(x, y):  
    return x - y  
  
def multiply(x, y):  
    return x * y  
  
def divide(x, y):  
    if y == 0:  
        return "Error: Division by zero"  
    return x / y  
  
def main():  
    print("Simple Calculator")  
    print("Select operation:")  
    print("1. Add")  
    print("2. Subtract")  
    print("3. Multiply")  
    print("4. Divide")  
  
    choice = input("Enter choice (1/2/3/4): ")  
  
    if choice not in ['1', '2', '3', '4']:  
        print("Invalid input")
```

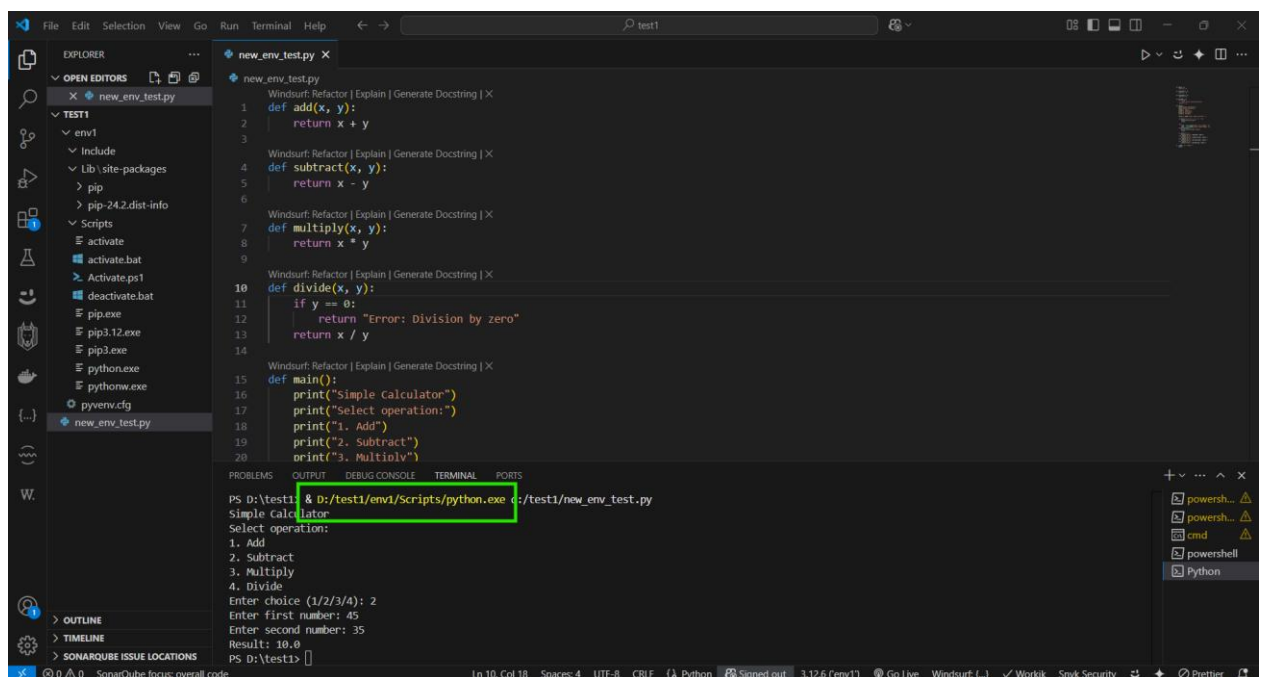
```
        return

    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
    except ValueError:
        print("Invalid number input")
        return

    if choice == '1':
        print(f"Result: {add(num1, num2)}")
    elif choice == '2':
        print(f"Result: {subtract(num1, num2)}")
    elif choice == '3':
        print(f"Result: {multiply(num1, num2)}")
    elif choice == '4':
        print(f"Result: {divide(num1, num2)}")

if __name__ == "__main__":
    main()
```

- A python script has been run in powershell with the selected interpreter env1 in VS Code, and outputting the result in the VS Code Terminal Panel.
- It runs the script and executes the code and produces an output.



Step 12: Activating the Virtual Environment

- In the cmd of VS Code, activate it by typing the command `env1\Scripts\activate`.
- Activating it will tell the system use the python interpreter that the virtual environment is being run from.

```
D:\test1>env1\Scripts\activate
```

After activating the environment with the ``env1\Scripts\activate`` command, the prompt changes to ``(env1) D:\test1>``.

Difference in Prompt:

- ``D:\test1>``: This is the standard command prompt, indicating the current directory (in this case, the "test1" project folder).
- ``(env1) D:\test1>``: This indicates that the virtual environment "env1" is active. When the environment is active, pip installs are directed to the environment.

```

1 @echo off
2
3 rem This file is UTF-8 encoded, so we need to update the current code page while executing it
4 for /f "tokens=2 delims=:." %%a in ("%SystemRoot%\System32\chcp.com") do (
5     set _OLD_CODEPAGE=%%a
6 )
7 if defined _OLD_CODEPAGE (
8     "%SystemRoot%\System32\chcp.com" 65001 > nul
9 )
10
11 set VIRTUAL_ENV=D:\test1\env1
12
13 if not defined PROMPT set PROMPT=$P$G
14
15 if defined _OLD_VIRTUAL_PROMPT set PROMPT=%_OLD_VIRTUAL_PROMPT%
16 if defined _OLD_VIRTUAL_PYTHONHOME set PYTHONHOME=%_OLD_VIRTUAL_PYTHONHOME%
17
18 set _OLD_VIRTUAL_PROMPT=%PROMPT%
19 set PROMPT=(env1) %PROMPT%
20
21 if defined PYTHONHOME set _OLD_VIRTUAL_PYTHONHOME=%PYTHONHOME%
22 set PYTHONHOME=
23
24 if defined _OLD_VIRTUAL_PATH set PATH=%_OLD_VIRTUAL_PATH%
  
```

```
(env1) D:\test1>
```



Step 13: Basic Requirements File

Creating a 'requirements.txt' file for installations. For this demonstration, pandas and numpy is being used.

```

1 pandas
2 numpy
  
```

```

pandas
numpy
  
```

Step 14: Installing from requirements.txt

- Shows `pip install -r requirements.txt` command.
- This is an efficient way to install multiple required packages for the project into the activated environment.
- It creates the `requirements.txt` file to contain the packages to be downloaded, and executes the `pip` command to install.

The screenshot shows an IDE with a file explorer on the left, a code editor at the top, and a terminal at the bottom. The file explorer shows a project structure with a folder 'TEST1' containing a subfolder 'env1'. Inside 'env1', there are files like 'activate', 'activate.bat', 'Activate.ps1', 'deactivate.bat', 'pip.exe', 'pip3.12.exe', 'pip3.exe', 'python.exe', 'pythonw.exe', and 'pyvenv.cfg'. A file named 'requirements.txt' is highlighted with a green box. The code editor shows the contents of 'requirements.txt' with two lines: 'pandas' and 'numpy'. The terminal shows the command '(env1) D:\test1>env1\Scripts\activate' and then '(env1) D:\test1>pip install -r requirements.txt'. The output of the command is shown below, indicating that pandas and numpy are being collected and installed. The command and its output are highlighted with green boxes.

```
new_env_test.py 1 pandas
activate.bat env1\Scripts 2 numpy

requirements.txt

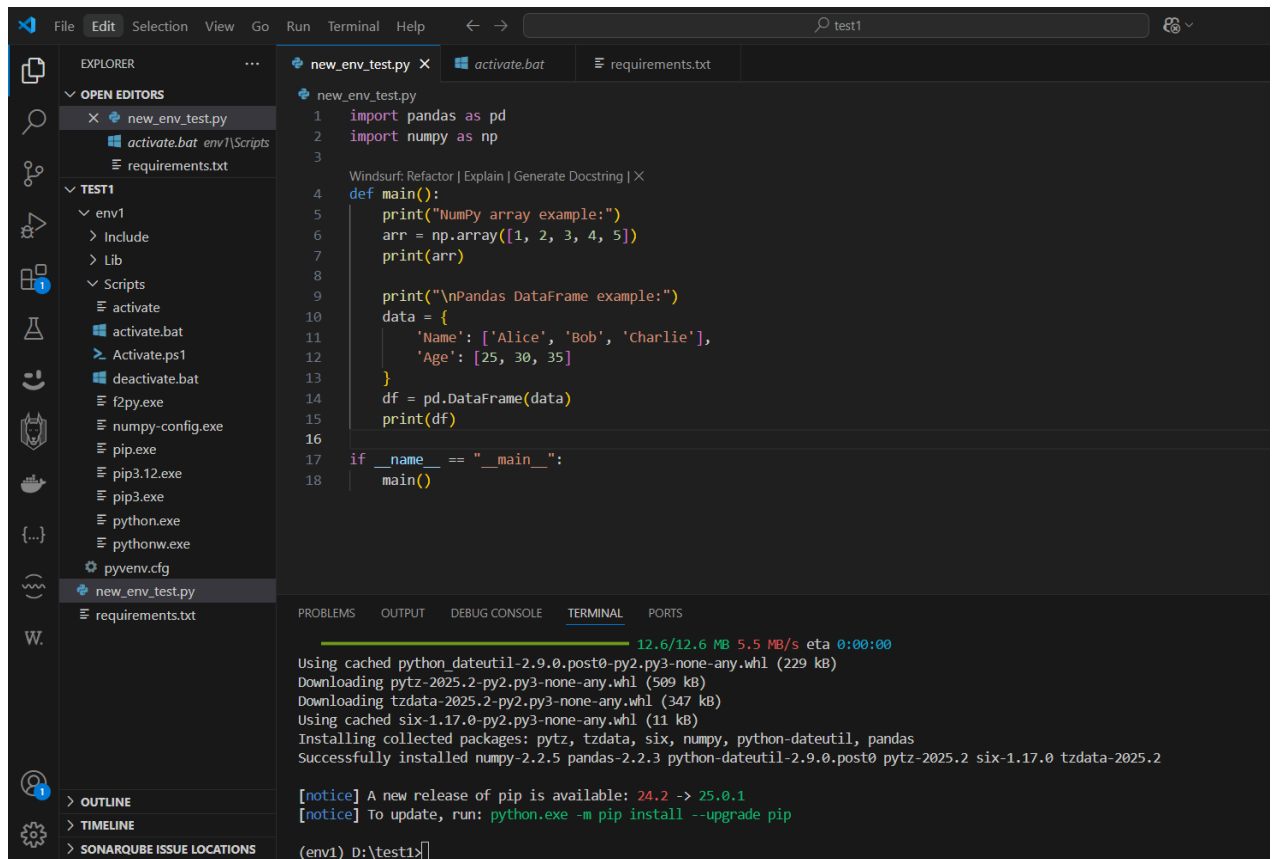
TEST1
├── env1
│   ├── Include
│   ├── Lib
│   └── Scripts
│       ├── activate
│       ├── activate.bat
│       ├── Activate.ps1
│       ├── deactivate.bat
│       ├── pip.exe
│       ├── pip3.12.exe
│       ├── pip3.exe
│       ├── python.exe
│       ├── pythonw.exe
│       └── pyvenv.cfg
└── new_env_test.py
    └── requirements.txt

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Microsoft Windows [Version 10.0.26100.3775]
(c) Microsoft Corporation. All rights reserved.

(env1) D:\test1>env1\Scripts\activate

(env1) D:\test1>pip install -r requirements.txt
Collecting pandas (from -r requirements.txt (line 1))
Using cached pandas-2.2.3-cp312-cp312-win_amd64.whl.metadata (19 kB)
Collecting numpy (from -r requirements.txt (line 2))
Downloading numpy-2.2.5-cp312-cp312-win_amd64.whl.metadata (60 kB)
Collecting python-dateutil>=2.8.2 (from pandas->-r requirements.txt (line 1))
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
```



The screenshot shows the Visual Studio Code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project named 'TEST1' with a subdirectory 'env1' containing various files. The code editor shows a Python script named 'new_env_test.py' with the following code:

```
1 import pandas as pd
2 import numpy as np
3
4 def main():
5     print("NumPy array example:")
6     arr = np.array([1, 2, 3, 4, 5])
7     print(arr)
8
9     print("\nPandas DataFrame example:")
10    data = {
11        'Name': ['Alice', 'Bob', 'Charlie'],
12        'Age': [25, 30, 35]
13    }
14    df = pd.DataFrame(data)
15    print(df)
16
17 if __name__ == "__main__":
18     main()
```

The terminal at the bottom shows the output of the script, including package installation progress and a notice about a new release of pip.



Step 15: Running a Python Script

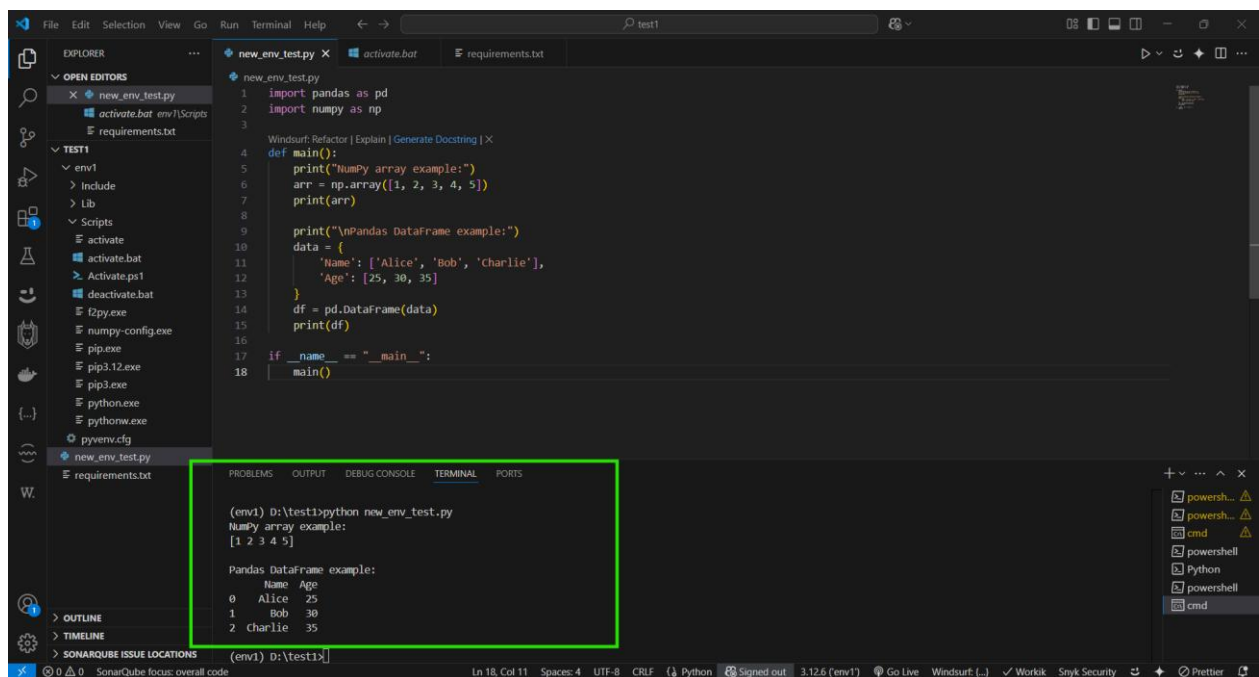
```
import pandas as pd
import numpy as np

def main():
    print("NumPy array example:")
    arr = np.array([1, 2, 3, 4, 5])
    print(arr)

    print("\nPandas DataFrame example:")
    data = {
        'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35]
    }
    df = pd.DataFrame(data)
    print(df)

if __name__ == "__main__":
    main()
```

- Successfully running a Python script that imports and uses pandas and numpy, after they were installed into the environment.
- It validates correct installation and functionality of the packages within the activated environment and runs the Python script "new_env_test.py" in the powershell of VS Code which in turn selects the correct python interpreter and is able to run numpy and pandas.

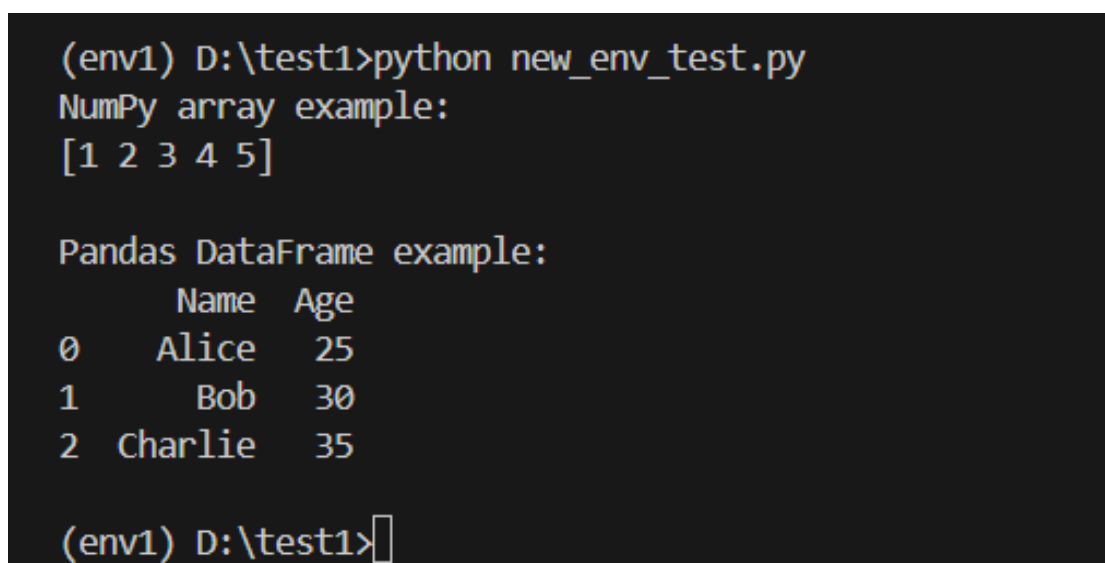


The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows the file structure with 'new_env_test.py' selected. The Editor pane shows the code for 'new_env_test.py', which imports pandas and numpy, creates a NumPy array, and a pandas DataFrame. The Terminal pane at the bottom shows the output of running the script in the 'env1' environment, displaying the NumPy array and the pandas DataFrame.

```
1 import pandas as pd
2 import numpy as np
3
4 def main():
5     print("NumPy array example:")
6     arr = np.array([1, 2, 3, 4, 5])
7     print(arr)
8
9     print("\nPandas DataFrame example:")
10    data = {
11        'Name': ['Alice', 'Bob', 'Charlie'],
12        'Age': [25, 30, 35]
13    }
14    df = pd.DataFrame(data)
15    print(df)
16
17 if __name__ == "__main__":
18     main()
```

```
(env1) D:\test1>python new_env_test.py
NumPy array example:
[1 2 3 4 5]

Pandas DataFrame example:
   Name  Age
0  Alice   25
1   Bob   30
2 Charlie  35
```



The terminal output shows the execution of the Python script in the 'env1' environment. It displays the NumPy array and the pandas DataFrame.

```
(env1) D:\test1>python new_env_test.py
NumPy array example:
[1 2 3 4 5]

Pandas DataFrame example:
   Name  Age
0  Alice   25
1   Bob   30
2 Charlie  35

(env1) D:\test1>
```